

# Vision3D Inspector

---

AI-powered platform for comparing **3D design models** with **real-world fabricated objects** using photographs, computer vision, photogrammetry, and 3D geometry analysis.

The platform allows users to upload a reference 3D model such as a `.gltf` file, capture or upload photographs of the physically manufactured object, and run an automated inspection to identify differences between the original design and the actual build.

---

## Overview

---

`vision3d-inspector` is designed for quality inspection of fabricated physical objects.

The basic workflow is:

```
graph TD; A[Upload 3D Reference Model] --> B[Capture / Upload Real Object Photos]; B --> C[Validate Images]; C --> D[Analyze Reference + Real Object]; D --> E[Detect Differences]; E --> F[Generate Inspection Results]; F --> G[Display Issues on 3D Viewer]; G --> H[Generate Inspection Report];
```

The system can be used for:

- Furniture inspection
- Exhibition fabrication
- Retail fixtures
- Interior installations
- Product prototypes

- Architectural mockups
  - Sculptures
  - Display installations
  - Custom manufacturing
  - Design-to-build quality checking
- 

## Main Goal

---

The goal of this platform is to answer:

How closely does the fabricated real-world object match the approved 3D design?

The system should identify problems such as:

- Missing components
  - Incorrect components
  - Geometry differences
  - Positioning problems
  - Alignment issues
  - Dimensional differences
  - Wrong proportions
  - Surface inconsistencies
  - Wrong colours
  - Material differences
  - Construction defects
  - Visible gaps
  - Finish problems
  - Damaged areas
- 

## Core Workflow

---

### 1. Create Project

---

A user creates a new inspection project.

Example:

Project Name:

Reception Counter Inspection

Description:

Compare fabricated reception counter with approved 3D design.

---

## 2. Upload Reference 3D Model

---

The user uploads the original design.

Initial supported format:

.glb

Future formats:

.gltf

.obj

.fbx

.stl

Non-GLB formats can eventually be converted internally to GLB.

---

## 3. View Reference Model

---

The uploaded model is displayed using an interactive 3D viewer.

The viewer should support:

- Rotate
  - Zoom
  - Pan
  - Reset camera
  - Fullscreen
  - Wireframe mode
  - Grid
  - Model dimensions
  - Issue markers
  - Deviation heatmap
-

# Real Object Capture

---

The user captures photographs of the physically fabricated object.

## Quick Inspection Mode

---

Requires approximately five guided photographs.

Recommended views:

1. Front
2. Front Left - 45°
3. Front Right - 45°
4. Rear
5. Top / Detail

Quick mode is primarily intended for visual inspection.

It can detect:

- Missing components
- Colour differences
- Material differences
- Visible alignment problems
- Surface defects
- Obvious geometry differences

---

## Precision Inspection Mode

---

Precision inspection uses:

20-40 photographs

or:

10-20 second walk-around video

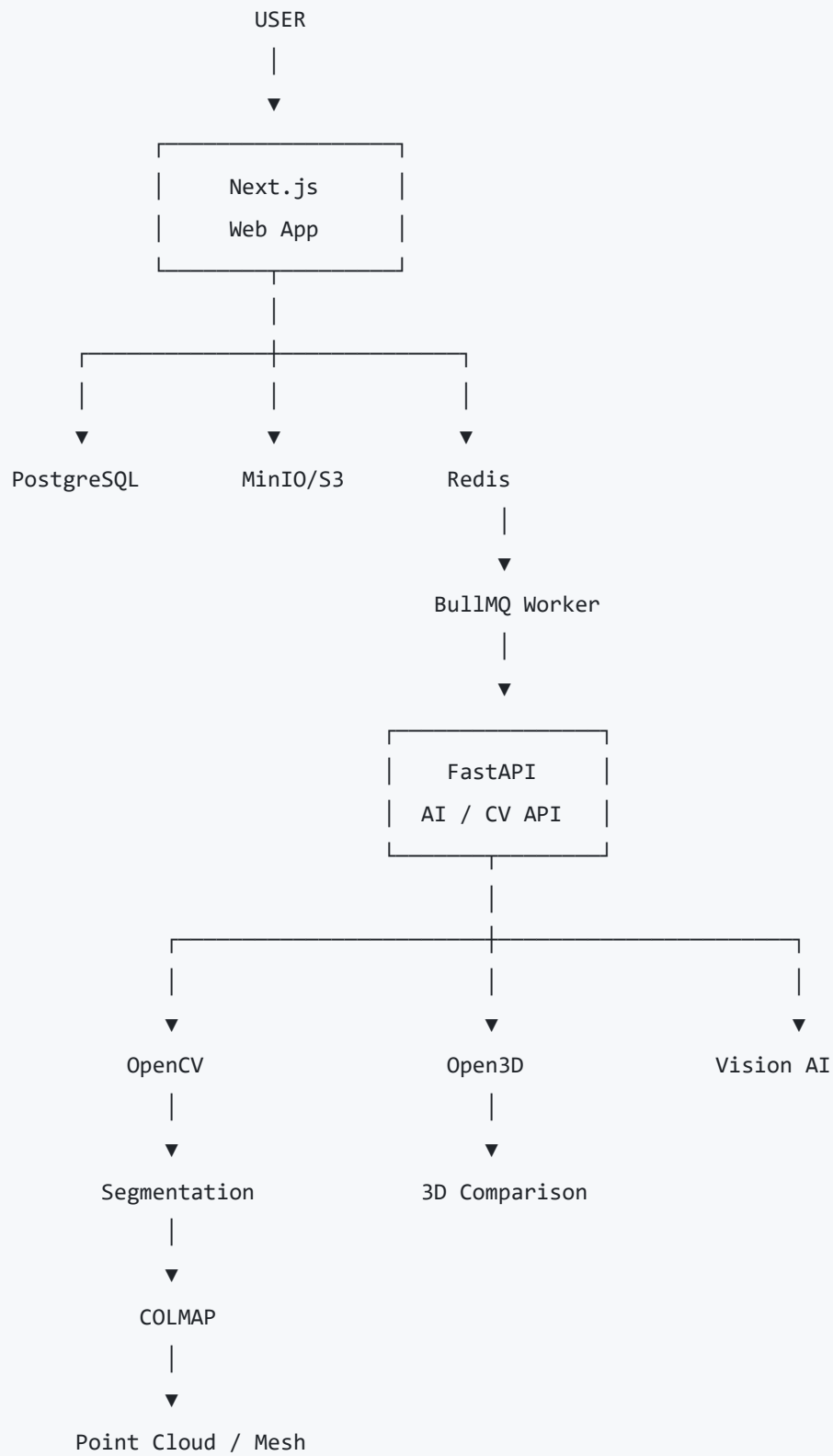
Video frames are extracted automatically and used for 3D reconstruction.

Precision mode can eventually provide:

- Point-cloud reconstruction
  - Camera pose estimation
  - 3D alignment
  - Surface comparison
  - Dimensional deviation
  - Geometry heatmaps
  - More accurate defect localization
-

# System Architecture

---



# Technology Stack

---

## Frontend

---

Next.js  
TypeScript  
React  
Tailwind CSS  
shadcn/ui

## 3D Viewer

---

Three.js  
React Three Fiber  
@react-three/drei

## Backend

---

Next.js Route Handlers  
Node.js  
Prisma  
PostgreSQL

## Storage

---

Development:

MinIO

Production:

AWS S3

Files stored include:

- GLB models
  - Uploaded images
  - Captured photos
  - Video
  - Segmented images
  - Point clouds
  - Generated meshes
  - Analysis JSON
  - Inspection reports
- 

## Background Processing

---

Long-running analysis should not execute inside a standard HTTP request.

The project uses:

Redis  
BullMQ

Analysis flow:

```
User clicks Analyze
      ↓
Next.js creates Analysis Job
      ↓
Job added to Redis
      ↓
BullMQ Worker receives job
      ↓
Python FastAPI processing
      ↓
Result stored in database
      ↓
Frontend displays results
```

---

# AI / Computer Vision Service

---

The AI service is implemented separately using Python.

Recommended stack:

```
Python
FastAPI
OpenCV
NumPy
Pillow
Trimesh
Open3D
COLMAP
SAM / segmentation model
Multimodal Vision AI
```

---

## Analysis Pipeline

---

The complete inspection pipeline will eventually look like:

## REFERENCE GLB

- |
- |— Validate model
- |— Extract dimensions
- |— Extract mesh
- |— Calculate bounding box
- |— Generate reference views



## REAL OBJECT PHOTOS

- |
- |— Image quality validation
- |— Blur detection
- |— Lighting validation
- |— Segmentation
- |— Calibration
- |— Camera pose estimation
- |— Reconstruction



## REAL POINT CLOUD



## REFERENCE POINT CLOUD



## GLOBAL REGISTRATION



## ICP



## GEOMETRY COMPARISON

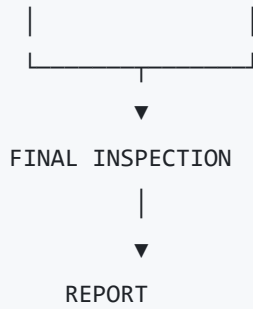


## DEVIATION MAP



## GEOMETRY ISSUES

## VISION ISSUES



---

## Inspection Types

---

The platform separates inspection into two engines.

### Geometry Engine

---

Used for measurable differences.

Examples:

- Position
- Dimensions
- Alignment
- Angle
- Distance
- Missing geometry
- Shape difference
- Surface deviation

Possible technologies:

- Open3D
- Trimesh
- COLMAP
- OpenCV

---

### Vision AI Engine

---

Used for visual and semantic inspection.

Examples:

Wrong material  
Wrong colour  
Missing trim  
Surface damage  
Scratch  
Poor finishing  
Visible construction problem  
Component mismatch

The geometry engine should be responsible for measurements.

The Vision AI should not invent dimensional measurements.

---

## Calibration

---

Precision measurement requires a known physical reference.

The recommended approach is to place an:

ArUco marker

near the physical object.

Example:

100mm × 100mm marker

The calibration marker helps establish:

- Physical scale
- Camera pose
- Pixel-to-real-world relationship
- Measurement reference

Without a known scale reference, results should be treated as visual or relative comparisons rather than exact millimetre measurements.

---

# Analysis Status

---

An analysis job can have the following states:

|                   |
|-------------------|
| QUEUED            |
| VALIDATING        |
| PROCESSING_IMAGES |
| SEGMENTING        |
| CALIBRATING       |
| RECONSTRUCTING    |
| ALIGNING          |
| COMPARING         |
| ANALYZING_VISUALS |
| GENERATING_REPORT |
| COMPLETED         |
| FAILED            |

---

# Example Analysis Progress

---



# Example Analysis Result

---

```
{
  "projectId": "project_001",
  "status": "COMPLETED",
  "issues": [
    {
      "id": "issue_001",
      "type": "GEOMETRY",
      "severity": "HIGH",
      "title": "Left panel misalignment",
      "description": "The fabricated left panel extends beyond the reference geometry.",
      "expectedValue": "420 mm",
      "actualValue": "446 mm",
      "deviation": 26,
      "confidence": 0.94,
      "position": {
        "x": 1.4,
        "y": 0.8,
        "z": 0.3
      }
    },
    {
      "id": "issue_002",
      "type": "MISSING_COMPONENT",
      "severity": "HIGH",
      "title": "Decorative trim missing",
      "description": "The decorative trim visible in the reference model was not
detected.",
      "confidence": 0.91
    }
  ]
}
```

---

## Issue Categories

Possible issue categories include:

GEOMETRY

DIMENSION

ALIGNMENT

POSITION

ANGLE

MISSING\_COMPONENT

WRONG\_COMPONENT

MATERIAL

COLOR

SURFACE\_FINISH

GAP

DAMAGE

SCRATCH

DEFORMATION

CONSTRUCTION

---

# Severity Levels

---

|          |
|----------|
| LOW      |
| MEDIUM   |
| HIGH     |
| CRITICAL |

---

# Repository Structure

---

```
vision3d-inspector/
|
├─ apps/
|
|   └─ web/
|       |
|       └─ app/
|           |
|           └─ components/
|               |
|               └─ lib/
|                   |
|                   └─ types/
|                       |
|                       └─ public/
|                           |
|                           └─ package.json
|                               |
|                               └─ Dockerfile
|
└─ ai-service/
    |
    └─ app/
        |
        └─ api/
            |
            └─ services/
                |
                └─ schemas/
                    |
                    └─ workers/
                        |
                        └─ utils/
                            |
                            └─ requirements.txt
                                |
                                └─ Dockerfile
|
├─ packages/
|
|   └─ database/
|       |
|       └─ prisma/
|           |
|           └─ schema.prisma
|               |
|               └─ client.ts
|
└─ shared-types/
```

```
| |
| └─ config/
|
|─ docker/
|
|─ docker-compose.yml
|
|─ package.json
|
|─ .env.example
|
└─ README.md
```

---

# Next.js Structure

---

apps/web/

```
├─ app/
│
│   ├─ (auth)/
│   │   ├─ login/
│   │   └─ register/
│   │
│   ├─ dashboard/
│   │
│   ├─ projects/
│   │   │
│   │   ├─ page.tsx
│   │   │
│   │   ├─ new/
│   │   │
│   │   └─ [projectId]/
│   │       │
│   │       ├─ page.tsx
│   │       ├─ model/
│   │       ├─ capture/
│   │       ├─ analysis/
│   │       └─ report/
│   │
│   └─ api/
│       │
│       ├─ projects/
│       ├─ models/
│       ├─ uploads/
│       ├─ captures/
│       ├─ analysis/
│       └─ reports/
│
├─ components/
│
│   ├─ ui/
│   │
│   └─ viewer/
│       ├─ ModelViewer.tsx
│       └─ ModelScene.tsx
```

```
| | └─ IssueMarker.tsx
| | └─ HeatMap.tsx
| | └─ MeasurementTool.tsx
| |
| └─ capture/
|   | └─ CameraCapture.tsx
|   | └─ CaptureGuide.tsx
|   | └─ ImageQuality.tsx
|   |
|   └─ analysis/
|       | └─ AnalysisSummary.tsx
|       | └─ AnalysisProgress.tsx
|       | └─ IssueCard.tsx
|       | └─ ComparisonViewer.tsx
|       |
|       └─ lib/
|           |
|           | └─ db.ts
|           | └─ storage.ts
|           | └─ queue.ts
|           | └─ auth.ts
|           | └─ validators.ts
|           | └─ ai-client.ts
|           |
|           └─ types/
```

---

# Python AI Service Structure

---

apps/ai-service/

```
|— app/
|
|   |— main.py
|
|   |— api/
|   |   |— health.py
|   |   |— analyze.py
|   |   |— reconstruction.py
|   |   |— comparison.py
|   |
|   |— services/
|   |   |— quality/
|   |   |   |— image_quality.py
|   |   |
|   |   |— segmentation/
|   |   |   |— segmentation.py
|   |   |
|   |   |— calibration/
|   |   |   |— aruco.py
|   |   |
|   |   |— reconstruction/
|   |   |   |— colmap.py
|   |   |
|   |   |— registration/
|   |   |   |— global_registration.py
|   |   |   |— icp.py
|   |   |
|   |   |— geometry/
|   |   |   |— mesh.py
|   |   |   |— dimensions.py
|   |   |   |— deviation.py
|   |   |
|   |   |— vision/
|   |   |   |— visual_inspection.py
|   |   |
|   |   |— reporting/
|   |   |   |— report.py
```

```
| |
| |─ schemas/
| |─ workers/
| |─ utils/
|
|─ requirements.txt
|─ Dockerfile
```

---

## Database Models

---

Initial database entities:

```
User

Project

ReferenceModel

CaptureSession

Photo

AnalysisJob

AnalysisResult

DetectedIssue

InspectionRule

Report
```

Relationship:

User

|

└ Project

|

└ ReferenceModel

|

└ CaptureSession

|

└ Photos

|

└ AnalysisJob

|

└ AnalysisResult

|

└ DetectedIssues

---

# Storage Structure

---

```
projects/
|
└─ {projectId}/
  |
  ├─ reference/
  |   │─ original.glb
  |   └─ optimized.glb
  |
  ├─ captures/
  |   └─ {captureId}/
  |       │─ front.jpg
  |       │─ front-left.jpg
  |       │─ front-right.jpg
  |       │─ rear.jpg
  |       └─ detail.jpg
  |
  ├─ segmentation/
  |
  ├─ reconstruction/
  |   │─ point-cloud.ply
  |   └─ reconstructed.glb
  |
  ├─ analysis/
  |   │─ issues.json
  |   └─ deviation.json
  |
  └─ reports/
      └─ inspection-report.pdf
```

---

## Development Roadmap

### Phase 1 — Platform Foundation

---

Build:

- Next.js application

- PostgreSQL
  - Prisma
  - Redis
  - MinIO
  - Docker Compose
  - Authentication
  - Project management
- 

## Phase 2 — 3D Model Support

---

Build:

- GLB upload
  - File validation
  - 3D viewer
  - Orbit controls
  - Bounding-box calculation
  - Model dimensions
  - Model metadata
- 

## Phase 3 — Photo Capture

---

Build:

- Camera access
  - Image upload
  - Guided capture
  - Front view
  - Left view
  - Right view
  - Rear view
  - Detail view
- 

## Phase 4 — Visual Inspection MVP

---

Build:

- Reference model screenshots
- Image comparison
- Multimodal AI analysis

- Structured issue JSON
- Analysis results page
- Severity system
- Issue cards

This will be the first usable MVP.

---

## Phase 5 — Image Processing

---

Add:

- Blur detection
  - Brightness validation
  - Exposure validation
  - Segmentation
  - Object isolation
  - ArUco marker detection
- 

## Phase 6 — Precision Inspection

---

Add:

- Multi-photo capture
  - Video capture
  - FFmpeg frame extraction
  - COLMAP reconstruction
  - Camera pose estimation
  - Dense point-cloud generation
- 

## Phase 7 — 3D Comparison

---

Add:

- GLB sampling
  - Point-cloud conversion
  - Global registration
  - ICP alignment
  - Nearest-surface comparison
  - Geometry deviation calculations
-

## Phase 8 — Error Visualization

---

Add:

- Issue clustering
  - 3D issue markers
  - Deviation heatmap
  - Click-to-focus
  - Reference vs real comparison
  - Tolerance rules
- 

## Phase 9 — Reporting

---

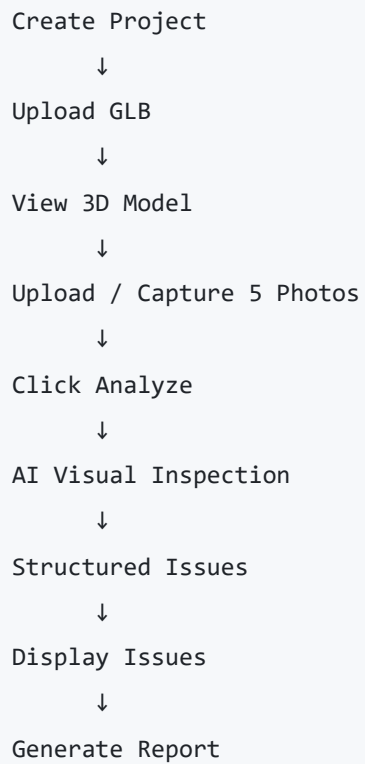
Add:

- Inspection summary
  - Issue screenshots
  - Reference screenshots
  - Actual photographs
  - Measurement data
  - PDF report generation
- 

## MVP Target

---

The first production milestone should support:



The MVP should focus on identifying:

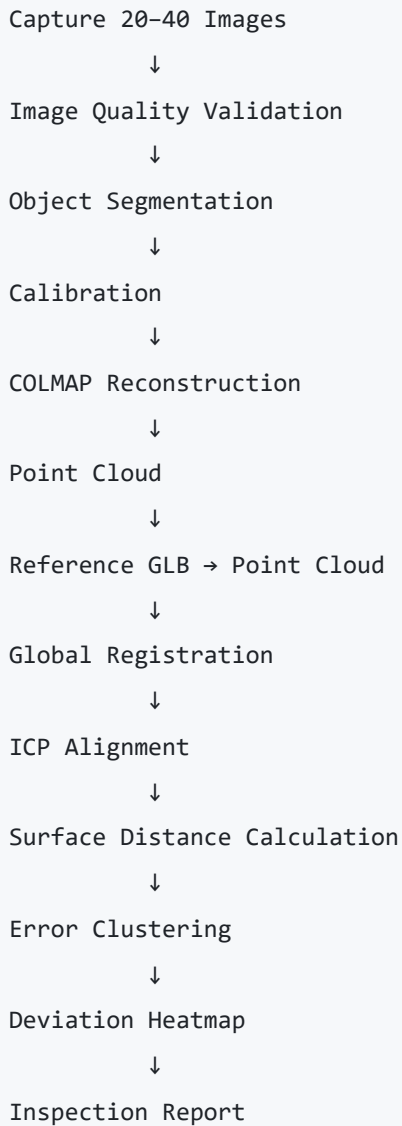
- Missing elements
- Wrong components
- Colour differences
- Material differences
- Visible geometry problems
- Alignment issues
- Surface problems
- Construction defects

Exact dimensional measurements should be added later through the precision inspection pipeline.

---

# Future Precision Workflow

---



---

## Local Development

### Prerequisites

---

Install:

```
Node.js  
npm  
Docker  
Docker Compose  
Python  
Git
```

For advanced reconstruction:

```
COLMAP  
FFmpeg
```

---

## Clone Repository

---

```
git clone <repository-url>  
cd vision3d-inspector
```

---

## Environment Variables

---

Create:

```
.env
```

from:

```
.env.example
```

Example:

```
DATABASE_URL=postgresql://postgres:postgres@localhost:5432/vision3d
```

```
REDIS_URL=redis://localhost:6379
```

```
MINIO_ENDPOINT=localhost
```

```
MINIO_PORT=9000
```

```
MINIO_ACCESS_KEY=minioadmin
```

```
MINIO_SECRET_KEY=minioadmin
```

```
MINIO_BUCKET=vision3d
```

```
AI_SERVICE_URL=http://localhost:8000
```

```
NEXT_PUBLIC_APP_URL=http://localhost:3000
```

Do not commit production secrets to Git.

---

## Start Infrastructure

---

```
docker compose up -d
```

Expected services:

PostgreSQL

Redis

MinIO

AI Service

## Install Dependencies

---

```
npm install
```

---

# Database Migration

---

```
npx prisma migrate dev
```

---

## Start Next.js

---

```
npm run dev
```

Application:

```
http://localhost:3000
```

---

## Start AI Service

---

From:

```
apps/ai-service
```

run:

```
uvicorn app.main:app --reload --port 8000
```

AI API:

```
http://localhost:8000
```

Health check:

```
GET /health
```

Expected response:

```
{  
  "status": "ok"  
}
```

---

## Recommended Implementation Order

---

Follow this exact development order:

01. Repository setup
02. Docker Compose
03. PostgreSQL
04. Prisma
05. Redis
06. MinIO
07. Authentication
08. Projects
09. GLB upload
10. Three.js viewer
11. Camera capture
12. Photo upload
13. Image quality checks
14. AnalysisJob table
15. BullMQ
16. FastAPI
17. Vision AI comparison
18. Structured issue output
19. Analysis UI
20. Issue markers
21. Reports

----- MVP COMPLETE -----

22. Segmentation
  23. ArUco calibration
  24. Video capture
  25. FFmpeg frame extraction
  26. COLMAP
  27. Point-cloud generation
  28. Trimesh
  29. Open3D registration
  30. ICP alignment
  31. Deviation calculation
  32. Error clustering
  33. Heatmap
  34. Tolerances
  35. Precision reporting
-

# Important Design Principle

---

Do not use a Vision LLM as the only inspection engine.

The platform should eventually use:

Computer Vision  
+  
Geometry Processing  
+  
Vision AI

Geometry processing should determine measurable differences.

Vision AI should explain and categorize visual differences.

---

## Project Status

---

Status: Initial Development  
Version: 0.1.0

Current priority:

MVP  
  
GLB Upload  
+  
3D Viewer  
+  
Photo Capture  
+  
Visual AI Analysis  
+  
Inspection Results

---

# Repository

---

vision3d-inspector

AI-powered 3D design-to-reality inspection platform.